

第8章 构造和析构

8.1 构造函数

构造函数用于在创建新对象时执行初始化操作，如初始化数据成员或执行其他自定义任务。作为一种特殊的成员函数，C++标准对构造函数有特定的语法要求：

- 构造函数的名称必须与类名相同。
- 可以带有参数列表，但不能有任何返回类型，即使是 `void` 也不行。
- 允许重载构造函数。
- 构造函数的访问控制可以是 `public`、`protected` 或 `private`。
- 可以使用 `explicit` 关键字修饰构造函数，但不能带 `const` 修饰，也不能是静态成员函数或常成员函数。

例如：

```
1 class MyClass {  
2     public:  
3         MyClass( );           //合法  
4         explicit MyClass(int n); //合法  
5         MyClass( YourClass & obj); //合法  
6     private:  
7         explicit MyClass(float val); //合法  
8 };
```

构造函数作为特殊的成员函数，自然有别于普通成员函数。首先，在语法方面，构造函数具有独特的格式要求。其次，尽管构造函数在实现上与普通实例方法相似，但在使用时，其行为和调用方式却兼具实例方法和类方法的特征。

例如，构造函数的第一个参数是隐式的 `this` 指针，这一点类似于实例方法。然而，构造函数不能带有 `const` 修饰，并且在还未创建任何对象时，就可以访问构造函数，这又类似于类方法。因此，在使用构造函数时存在一些特殊性，如类方法中不能直接访问实例方法，但可以调用构造函数。

8.1.1 自定义构造函数

程序员可以通过指定参数列表，`explicit` 关键字，访问控制等自定义类的构造函数。

构造函数是可以重载的，例如 `MyClass` 类中，重载了多个自定义的构造函数。

构造函数的参数列表可以是不同的，也可以指定参数的缺省值。但不要出现二义性问题，也称

模棱两可错误。例如：

```

1  class MyClass {
2  public:
3      MyClass( ) { mN1 = 0; mN2 = 0; }
4      MyClass( int v1 ) { mN1 = v1; mN2 = 0; }
5      MyClass( int v1, int v2 = 0 ) { mN1 = v1; mN2 = v2; }
6  private:
7      int mN1;
8      int mN2;
9  };
10 int main( ) {
11     MyClass obj1;           //语句 1,合法
12     MyClass obj2(2);        //语句 2,二义性错误
13     MyClass obj3(3,4);      //语句 3,合法
14 }
```

由于参数个数不同，重载三个构造函数是合法的。语句 1 会调用 `MyClass` 中的无参构造函数，语句 3 会调用带有两个参数的构造函数，而语句 2 将导致二义性错误。对于 `MyClass obj2(2);`，编译器无法确定是调用 `MyClass(2)` 还是 `MyClass(2, 0)`。对于由默认参数值引起的此类二义性错误，可以通过以下方式修改函数定义：

```

1  class MyClass {
2  public:
3      MyClass( int v1 = 0, int v2 = 0 ):mN1(v1),mN2(v2) { }
4  private:
5      int mN1;
6      int mN2;
7  };
8  int main( ) {
9      MyClass obj1;           // 编译成 MyClass obj1(0,0);
10     MyClass obj2(2);        // 编译成 MyClass obj2(2,0);
11     MyClass obj3(3,4);      // 编译成 MyClass obj3(3,4);
12 }
```

这相当于提供了三个重载的构造函数：无参构造函数、带一个参数的构造函数以及带两个参数的构造函数。

构造函数可以使用 `explicit` 关键字修饰。`explicit` 关键字表明，该构造函数必须由程序员在代码中显式调用，编译器在编译过程中不会隐式调用此构造函数。例如：

```

1  class MyClass {
2  public:
```

```

3     explicit MyClass( int v1, int v2 = 2 ) { mN1 = v1; mN2 = v2; }
4 private:
5     int mN1;
6     int mN2;
7 };
8 void func( const MyClass & obj) { }
9 int main() {
10     MyClass obj1( 3 );           // 显式调用 MyClass::MyClass(3,2)
11     MyClass obj2( 4, 5 );        // 显式调用 MyClass::MyClass(4,5)
12     MyClass obj3 { 6 };          // 显式调用 MyClass::MyClass(6,2)
13     MyClass obj4 { 7, 8 };       // 显式调用 MyClass::MyClass(7,8)
14     MyClass obj5 = { 8, 9 };     // 非法,隐式调用 MyClass::MyClass(8,9)
15     func( obj1 );                // 合法
16     func( obj2 );                // 合法
17     func( 100 );                 // 非法,隐式调用 MyClass::MyClass(100,2)
18 }

```

这里, obj1、obj2 使用直接初始化进行构造, obj3、obj4 使用列表初始化进行构造, 而 obj5 使用复制列表初始化进行构造, 这需要隐式调用构造函数, 因此非法; 同理, 调用 func(100) 时, 需要将整数 100 隐式转换为 MyClass 对象, 由于构造函数被标记为 explicit, 禁止了这种隐式转换, 调用也是非法的。如果移除 explicit 关键字, obj5 的构造和 func(100) 都会成为合法。

此外, 构造函数可以具有不同的访问控制级别。最常见的是 public 访问控制, 这允许在类的外部实例化该类的对象。当然, 也可以使用其他访问控制级别, 例如:

```

1 class Name {
2 public:
3     static Name & create( ) {
4         static Name obj;
5         return obj;
6     }
7 private:
8     Name( ) { }
9 };
10 int main( ) {
11     Name & obj = Name::create( );
12 }

```

上述代码实际是单例模式的部分实现, 可确保单例的 Name 类的对象最多只有一个实例存在。通过将构造函数设为私有, 并仅在公有的类方法 create 中调用, 保证了 Name 对象只能通过 create 方法获得。

8.1.2 缺省构造函数

类中也可以不定义自定义构造函数,在这种情况下,编译器会自动提供一个缺省的构造函数,也称预设的或默认的构造函数。这个缺省构造函数是无参且公有的。需要注意的是,编译器仅在类中没有任何自定义构造函数时才会提供这个缺省构造函数。例如,对于类 A 而言,以下情形都会导致在 main 函数中实例化对象失败。

```

1  class A {
2  public:
3      A(int n) { }      //自定义一个有参构造函数
4  };
5  class A {
6  public:
7      A( ) = delete;    //禁用了构造函数,也属于自定义
8  };
9  class A {
10 private:
11     A( ) { }          //定义私有的无参构造函数
12 };
13 class A{
14 public:
15     A(const A & ) { } //自定义拷贝构造函数,也属于自定义构造函数
16 };
17 int main( ) {
18     A a; //对上面几种情况,此语句都将失败
19 }
```

8.1.3 对象的初始化

构造函数的主要职责是在创建对象时初始化其实例变量,但不涉及类变量的初始化。这一过程称为对象的初始化,其目的是在实例化时为各个实例变量设定初始值。

在 C++ 中,初始化 T 类型对象时,可以采用下列几种形式之一:

```

格式1:  T obj ;
格式2:  T obj = other ;
格式3:  T obj = { value1,value2,value3,... } ;
格式4:  T obj ( value1,value2,value3,... ) ;
格式5:  T obj { value1,value2,value3,... } ;
```

其中格式 1 中没有指定初始化参数,称为缺省(默认)初始化。格式 2 和格式 3 使用了=,称为拷贝初始化(copy-initialize)。格式 4 和格式 5 没有使用=,称为直接初始化(direct-initialize)。无论哪种

形式，都会调用参数匹配的构造函数。拷贝初始化和直接初始化的主要区别在于，拷贝初始化不会调用使用 `explicit` 修饰的构造函数。例如：

```

1  #include <string>
2  struct Point {
3      int x;
4      int y;
5  };
6  class Dog {
7  public:
8      Dog(const std::string & name, int age )
9          : mName(name), mAge(age)
10         { }
11      explicit Dog( int age )
12          : mName( "未知" ), mAge( age )
13         {}
14  private:
15      std::string mName;
16      int      mAge;
17  };
18  int main( ) {
19      Point  pt;           //合法,x,y 值不定
20      //Point pt1 ( 1, 2 ); //非法,无相应的构造函数
21      Point pt3 = { 1,2 }; //合法, 拷贝列表初始化
22      Point pt2 { 1, 2 };  //合法, 直接列表初始化
23
24      Dog dog1( "大黄", 3 ); //合法, 直接初始化
25      Dog dog2 = { "大黄", 4 }; //合法, 拷贝列表初始化
26      Dog dog3 { "大黄", 5 }; //合法, 直接列表初始化
27      Dog dog4( 6 ); //合法, 直接初始化
28      //Dog dog4 = {6}; //非法, 拷贝列表初始化, 不能调用 explicit 构造函数
29      Dog dog6 { 6 }; //合法, 直接列表初始化, 可以调用 explicit 构造函数
30  }

```

使用花括号{ 和 }的初始化形式是 C++11 中引入的，相较于圆括号(和)，可以有效避免构造过程中参数的窄化问题以及明确区分声明和定义。因此，无论是对象的初始化还是数据成员的初始化，使用花括号{ 和 }都是标准中推荐的做法。例如：

```

1  class Dog {
2  public:
3      Dog(const std::string & name = "未知", int age=1 )
4          : mName(name), mAge(age)
5          { }

```

```

6  private:
7      std::string mName;
8      int      mAge;
9  };
10 int main( ) {
11     Dog dog1("大黄", 3);
12     Dog dog2{"大黄", 3};
13
14     Dog dog3("大黄", 3.7); //通常只给出编译警告，不作为编译错误
15     Dog dog4{"大黄", 3.7}; //给出精度丢失的编译错误
16
17     Dog dog5( ); //dog5 是函数声明！不是定义！
18     Dog dog6{ }; //dog6 明确是一个对象的定义
19 }

```

8.1.4 初值的指定

构造函数用于创建对象，为了在创建的同时指定对象中数据成员的初始值，可以采用缺省初始化、类内初始化以及成员初始化列表初始化。

缺省初始化

构造对象时，如果未指定构造参数，会执行无参或参数均有缺省值的构造函数。如果该构造函数是用户自定义的，则按用户定义进行初始化；若用户没有提供，则使用编译器提供的缺省构造函数，即进行缺省(默认)的初始化。此时，对于成员变量：

- 如果在类定义中已指定初始值，则按指定值进行初始化；
- 如果未指定初始值，则根据变量的类型使用相应的构造函数进行初始化。对于内置类型(如 int、float 等)，其初始值取决于存储位置，全局或静态变量通常会被初始化为 0，而栈上的局部变量则可能是未定义的值；对于类类型的成员，将调用该类的无参构造函数进行初始化。

例如：

```

1  class Data {
2  public:
3      Data(int n = 7 ) { mN = n;}
4  private:
5      int mN;
6  };
7  struct A {
8      int mInt = 10;

```

```

9      int mAny;
10     Data mData;
11 };
12 int main() {
13     A a;
14 }

```

在实例化对象 `a` 时,调用类 `A` 的无参构造函数,即编译器提供的缺省构造函数。构造完成后,对象 `a` 的数据状态如下: `mInt` 的值为 10, `mAny` 的值未定义,而 `mData` 对象中的 `mN` 值为 7。

类内初始化

如果自定义构造函数提供了各成员的初始值,则按照构造函数的要求进行初始化;此外还可以在定义类时,可以为实例变量指定缺省值或默认值,也称类内初始化。这样,在实例化对象时,如果没有提供初始值,则使用定义时指定的缺省值进行初始化。这种方式使得为实例变量设置缺省初始值变得非常方便。然而,对于类变量,通常只有当类变量可安全地转成整型且为常量(`const`)时,才能直接指定默认值,如 `long long`、`long`、`int`、`bool`、`char`、`wchar_t` 类型等。例如:

```

1  class Data {
2  public:
3      Data(int n ) {}
4  };
5  struct Point {
6      int x;
7      int y;
8  };
9  class A {
10 private:
11     int mA = 1;                //合法
12     float mVal = 0.0f;         //合法
13     A * mpA = nullptr;         //合法
14     Data mData = Data(888);    //合法
15     Point mPoint {1,2};        //合法
16     static const int NUM = 5;   //合法
17     static const unsigned int UNUM = 6; //合法
18     static const unsigned long LNUM = 7; //合法
19
20     //static int NUM2 = 5;       //非法
21     inline static int NUM3 = 5;  //C++17 以上才可以
22     //static const double PI = 3.1415926; //非法
23 };

```

成员初始化列表初始化

请看下面的代码：

```
1  class Card {
2  public:
3      Card( int x,int y) { mX = x; mY = y; }
4  private:
5      int mX;
6      int mY;
7  };
```

当构造函数执行到左花括号{时，表示对象已经创建完毕，花括号内部的代码都是修改对象！所以，这里的 `mX=x;mY=y`；实际上是赋值操作，不是初始化。虽然，从结果上效果是一样的，但由于赋值是在对象的所有实例变量已经分配完存储空间之后，再通过修改存储值来实现的，因此其效率较低；更重要的是，对于某些实例变量，不允许分配空间后再进行赋值操作。例如：

```
1  class Card {
2  public:
3      Card( int id, Player & player ) {
4          mID = id;           //非法
5          mOwner = player; //非法
6      }
7  private:
8      const int mID;
9      Player & mOwner;      //不建议使用引用型数据成员
10 };
```

为了解决此类问题，可以使用成员初始化列表来指定初始化方式。其语法格式为：

构造函数名(参数列表) : 成员初始化列表 { 函数实现 }

成员初始化列表由若干个用逗号分隔的初始化表达式组成。例如，上述 `Card` 类的定义可以修改为：

```
1  class Card {
2  public:
3      Card( int x, int y,int id, Player & player )
4          : mX(x),mY(y),mID(id),mOwner(player)
5          { }
6  private:
7      int mX;
8      int mY;
```

```

9      const int mID;
10     Player & mOwner; //不建议使用引用型数据成员
11 };

```

在使用成员初始化列表时，建议按照实例变量的定义顺序依次指定初始值，否则现代编译器通常会给出警告。此外，对于以下几种类型的实例变量，要么使用类内初始化，要么必须使用成员初始化列表进行初始化：

- 带有 `const` 修饰的数据成员；
- 引用类型的数据成员；
- 自定义类型的对象型数据成员，且该自定义类型没有无参构造函数。

例如：

```

1  class Description {
2  public:
3      Description( int n ) { }
4  };
5  class Player { };
6
7
8  class Card {
9  public:
10     Card( int aId, Player & aPlayer )
11         :mX( 99 ), mY(88), mID( aId ), mOwner( aPlayer ), mDesc( aId ) { }
12 private:
13     int      mX;
14     int      mY;
15     const int mID;
16     Player & mOwner; //不建议引用型数据成员
17     Description mDesc;
18 };

```

由于没有采用类内初始化，数据成员 `mID`、`mOwner` 和 `mDesc` 必须通过初始化列表进行初始化。而 `mX` 和 `mY` 既可以通过初始化列表初始化，也可以在构造函数的实现中通过赋值来设置值。然而，通常建议如果使用了初始化列表，就应该按照实例变量定义的顺序，在列表中为所有实例变量指定初始值，而不是部分使用初始化列表，另一部分不使用。

数值初始化

对于类类型来说，其初值总可以通过构造函数或类内初值指定。但对于内置类型或指针来说，如局部变量和类的数据成员，也可能需要指定初值，这时可以使用数值初始化。例如：

```

1  int * pi1 = new int;           //合法, 但 *pi1 的值不定
2  int * pi2 = new int( );       //合法, 且 *pi2 的值为 0
3  int * pi3 = new int { };     //合法, 且 *pi3 的值为 0
4  int * pi4 = new int( 2 );     //合法, 且 *pi4 的值为 2
5  int * pi5 = new int { 3 };   //合法, 且 *pi5 的值为 3
6  double v1;                   //合法, 但 v1 的值不定
7  double v2 = 1.0;             //合法, 且 v2 的值为 1.0
8  double v3 { };               //合法, 且 v3 的值为 0.0
9  double v4 { 3.14 };          //合法, 且 v4 的值为 3.14
10
11 int a1[ 5 ];                 //合法, 但数组中 5 项的值都不定
12 int a2[ 5 ] = { };           //合法, 且值为 0, 0, 0, 0, 0
13 int a3[ 5 ] { };             //合法, 且值为 0, 0, 0, 0, 0
14 int a4[ 5 ] = { 1,2 };       //合法, 且值为 1, 2, 0, 0, 0
15 int a5[ 5 ] { 3,4 };         //合法, 且值为 3, 4, 0, 0, 0
16 int a6[ 5 ] = { 5,4,3,2,1 }; //合法, 且值为 5, 4, 3, 2, 1
17 int a7[ 5 ] { 5,4,3,2,1 };   //合法, 且值为 5, 4, 3, 2, 1

```

8.1.5 构造函数的执行过

构造函数的主要任务是初始化各个数据成员, 其执行过程大致如下:

1. 为各非静态成员(包括实例变量和实例常量)分配空间, 并按照定义顺序依次进行初始化。
2. 初始化时, 首先检查初成员始化列表中是否指定了初始化方式。如果指定了, 则按指定的方式进行初始化。
3. 如果成员初始化列表中未指定, 检查是否指定了类内初始化初值。如果指定了, 按类内初始化进行。
4. 如果也没有指定类内初值, 则执行缺省初始化。
5. 按照上述步骤依次初始化所有非静态成员, 直到所有非静态成员都已分配并初始化。
6. 最后, 进入构造函数的函数体, 执行用户自定义的操作, 需要强调此处对数据成员的修改都是赋值修改。

例如:

```

1  class Card {
2  public:
3      Card( int aId, Player & aPlayer )
4          :mX( 99 ), mID( aId ), mOwner( aPlayer ), mDesc( aId ) { mY = 88; }
5  private:
6      int      mX;
7      int      mY;

```

```

8      const int    mID;
9      Player  &    mOwner;  //不建议引用型数据成员
10     Description mDesc;
11 };

```

在实例化一个 Card 对象时, 首先使用初始化列表按定义顺序构建实例变量: mX 初始化为 99, mY 的初值不定, mID 初始化为参数 aId, mOwner 初始化为 aPlayer, 最后 mDesc 使用 Description 类的单参构造函数进行初始化。之后, 进入构造函数的函数体, 将 mY 的值修改为 88。至此, 对象构造完成。

8.1.6 构造函数的特殊形式

在 C++11 中, 自定义构造函数增加了两种新的语法格式, 可以简化代码的编写。

格式 1: `T( ) = default;`

格式 2: `T( [参数列表] ) = delete;`

格式 1 表示, 自定义了一个无参构造函数, 但该构造函数的实现实际上是调用了编译器提供的缺省构造函数。例如:

```

1  class Dog {
2  public:
3      Dog( ) = default;
4      Dog( const string & name );
5      Dog( int age, const string & name );
6  };

```

这里定义了两个带参数的构造函数, 因此编译器将不再提供无参构造函数。如果仍然需要无参构造函数, 上边的写法最简单。

格式 2 表示禁用某个构造函数, 但这仍然被视为自定义了构造函数, 此时, 编译器不再提供缺省的无参构造函数。

8.1.7 委托构造

构造函数可以重载, 但在一个构造函数的函数体内不允许调用另一个构造函数, 这与普通成员函数的行为不同。因此, 在重载多个构造函数时, 可能会导致相同的代码块出现在多个构造函数中, 这显然不是良好的编程风格。为此, C++11 标准引入了委托构造来解决这一问题。

委托构造允许在一个构造函数的成员初始化列表中指定另一个构造函数。也就是说, 在构造对象时, 可以先将构造过程委托给其他构造函数完成初步构建, 然后再在当前构造函数的函数体内做

进一步的定制和修改。例如：

```

1  #include <string>
2  class Dog;
3  class AA {
4  public:
5      //使用了委托构造
6      AA( int n ) :AA( n, "某某" ) { }
7      //使用了委托构造
8      AA( int n, int id ) :AA( n , std::to_string( id ) ) { mNum = n; }
9      //普通构造函数
10     AA( int n, const string & str )
11         :mNum( n + 100 ), mName( str )
12     { }
13     //普通构造函数
14     AA( const Dog & dog ) { }
15 private:
16     int     mNum;
17     string mName;
18 };
19 int main( ) {
20     AA a1( 1 );           //mNum=101,mName="某某"
21     AA a2( 2, 10001 );    //mNum=2,mName="10001"
22     AA a3( 3, "张三" );   //mNum=103,mName="张三"
23 }

```

8.2 析构函数

析构函数与构造函数的作用相反，它在对象销毁时执行特定操作，为对象提供最后一次执行的机会，主要用于清理工作。

8.2.1 自定义析构函数

定义类时，可以自定义析构函数。析构函数的名称必须是“~”加类名，隐含第一个参数是 `this` 指针，不接受显式参数，也不返回任何值，并且析构函数不能使用 `const` 或 `static` 修饰，因此析构函数中 `this` 参数的类型只能是 `ClassName * const`。此外，由于没有显式参数，析构函数无法被重载，每个类只能拥有一个析构函数。析构函数的语法格式为：

格式1: `~ClassName( ) [noexcept];`

格式2: `~ClassName( ) [noexcept] { /* 自定义析构函数的实现 */ }`

自定义析构函数的访问控制通常为 `public`。虽然语法上允许将其声明为 `protected` 或 `private`,

但这种设计仅在极少数特殊场景下才有意义。若析构函数为 `protected` 或 `private`，则类外部将无法合法调用它，从而可能导致编译错误。因此若无明确目的，不建议将析构函数的访问控制设为非 `public`。例如：

```
1  class MyClass {
2  private: //私有的析构函数
3      ~MyClass( ) { }
4  };
5  int  main( ) {
6      MyClass obj;
7  } //main 结束时，自动释放 obj 对象，要调用 MyClass 类的析构函数，但类外禁止访问!! 编译报错!!!
```

析构函数的调用时机由系统严格控制：对于栈对象，析构发生在作用域结束时；对于堆对象，则在 `delete` 或 `delete[]` 执行时触发。因此除非使用定位 `new` 配合显式析构的高级场景，一般不允许用户显式地调用析构函数，否则可能引发未定义行为（如重复析构、资源双重释放等）。

在执行析构过程中，严格遵循以下顺序：

- (1) 执行析构函数体内的代码：实现用户自定义的清理逻辑；
- (2) 逆序销毁非静态数据成员：按照类中数据成员的定义顺序，从后往前依次调用其析构函数。若数据为类类型对象，则自动调用对应类型的析构函数；若数据为内置类型(如 `int`、`double`)、指针、引用、平凡数据 (POD / Plain Old Data) 时，这些数据不需资源清理，因此也不会执行任何析构操作，仅在后续释放内存时随对象内存一同被释放；
- (3) 自动执行基类的析构：存在继承时，先析构本类，再析构基类；
- (4) 最后，释放对象所占内存：对于堆区对象，由 `operator delete` 完成；对于栈区对象，由栈的回退自动完成；

例如：

```
1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      ~A( ) { cout << "析构 A! " << endl; }
6  };
7  class Description {
8  public:
9      Description( int n ) { }
10     ~Description( ) { cout << "析构 Description!" << endl; }
```

```

11 };
12 class Card {
13 public:
14     //构造函数
15     Card( int id ) :mID( id ), mDesc( id ) { cout << "构造 Card!" << endl; }
16     //析构函数
17     ~Card( ) { cout << "当前 mID = " << mID << " ,析构中 Card!" << endl; }
18     int getID( ) const { return mID; }
19 private:
20     const int     mID;
21     A             mA;
22     Description   mDesc;
23 };
24 int main( ) {
25     Card c1( 5 );
26     cout << c1.getID( ) << endl;
27 }

```

运行结果:

```

构造 Card!
5
当前 mID = 5 ,析构中 Card!
析构 Description!
析构 A!

```

8.2.2 缺省(默认)的析构函数

定义类时,如果用户未显式定义析构函数,那么编译器会自动提供一个缺省的析构函数。该缺省析构函数不带 `const` 或 `static` 修饰,其访问控制是 `public`。

缺省析构函数与自定义析构函数的区别在于析构的执行过程的第一步:自定义析构函数可以自定义析构过程中的清理逻辑,而缺省析构函数无法自定义。对于执行过程的后续步骤,两者同样都会执行。因此,缺省析构函数与自定义析构函数的核心区别在于:是否允许用户介入并控制析构的清理逻辑。

- 若定义了自定义析构函数,编译器将执行该函数体中的用户代码,用于实现释放资源、关闭句柄、解除注册等清理逻辑,随后仍会自动完成后续的析构过程-析构基类、析构数据、释放内存。
- 若使用缺省析构函数,编译器将跳过用户自定义的清理逻辑,但仍严格按标准顺序执行后续的析构过程。

8.3 对象的创建和销毁

构造函数用于对象创建时的初始化，而析构函数则用于对象销毁前的清理工作。在对象的生命周期中，从使用构造函数成功创建对象开始，到销毁对象之前，对象一直处于存活期，期间可以与其它对象进行对象间的相互通信，实现程序逻辑。这里要强调的是，在 C++ 中，对象的销毁不总是自动的。一旦程序结束前没有销毁已创建的对象，那么会造成内存的安全问题。因此必须知道什么时候会触发对象的创建和销毁。其中：

会触发调用构造函数的情况有：

- (1) 对象进行缺省初始化。
- (2) 对象进行指定参数的初始化。
- (3) 对象进行列表初始化。
- (4) 通过转换构造函数执行类型转换。
- (5) 构建对象型数据成员。

会触发调用析构函数的情况有：

- (1) 栈上或数据区的对象在其生命周期结束时，自动调用析构函数
- (2) 堆上通过 `new` 表达式创建的对象，在使用 `delete` 表达式释放时会调用析构函数
- (3) 显式调用析构函数。通常不允许用户直接以 `~T()` 的形式显式地调用析构函数，但在某些特殊情况下(例如使用 `placement new` 进行定位分配)是可以的。此外，还要注意，显式调用析构函数只会执行析构函数的函数体，但不会释放对象本身。

例如：

```

1  class B {
2  public:
3      B( ) { }
4      B( int n ) { }
5  };
6  class A {
7  public:
8      A( );
9      A( int n );
10     void method( const B & aB ) { }
11     ~A( );
12 private:
13     int     mNum;
14     B       mB1;

```

```

15     B      mB2;
16     static B  sGlobalB;
17 };
18 B A::sGlobalB( 200 );    //静态对象, 存放在数据区, 调用 B::B(200)构造
19 A::A( ) : mNum( 0 ), mB2( 2 ) { }
20 A::A( int n ) : mNum( n ), mB1( n ), mB2( n + 1 ) { }
21 int main( ) {
22     A a1;                //缺省初始化, 调用 A::A( ), 期间又调用 B::B( )和 B::B(2)
23     A a2( 2 );           //直接初始化, 调用 A::A( 2 ), 期间又调用 B::B(2)和 B::B(3)
24     A a3 { 3 };          //直接列表初始化, 调用 A::A( 3 ), 期间又调用 B::B(3)和 B::B(4)
25     a3.method( 100 );    //隐式转换, 访问构造函数 B::B(100)
26     A * pA1 = new A( );   //值初始化+缺省初始化, 调用 A::A( ), 期间又调用 B::B( )和 B::B(2)
27     A * pA2 = new A( 4 ); //值初始化+直接初始化, 调用 A::A( 4 ), 期间又调用 B::B(4)和 B::B(5)
28     A * pA3 = new A { 5 }; //值初始化+直接列表初始化, 调用 A::A( 5 ), 又调用 B::B(5)和 B::B(6)
29     delete pA3; //释放堆区对象, 调用 A::~~A( ), 期间两次调用 B::~~B( )
30     delete pA2; //释放堆区对象, 调用 A::~~A( ), 期间两次调用 B::~~B( )
31     delete pA1; //释放堆区对象, 调用 A::~~A( ), 期间两次调用 B::~~B( )
32 }

```

当 main 函数结束时, 会进行清栈操作, 栈上的三个对象将依次被释放。首先释放 a3 对象, 此时会调用类 A 的析构函数, 随后依次释放 a3 中的子对象 mB2 和 mB1, 每个子对象都会调用一次类 B 的析构函数。a3 释放完成后, 按照相同的过程依次释放 a2 和 a1。清栈完成后, 程序将结束并释放内存, 此时数据区中的全局对象 sGlobalB 也会被释放, 并再次调用类 B 的析构函数。